

Unit-6-Strings

PAGE No.

DATE

String Input/output function:-

- It is easy to read or write string of characters
- There are two types of functions gets and puts.

=> Gets():-

- The gets function is used to read a set of numeric character from the standard input until a return key is pressed and place a null in memory and written.
- The syntax of gets function is
`gets(variable-name)`

=> Puts():-

- The puts function continuous to send characters to the standard output from memory that it from the array. We declare until it run into the null. When it sends a new and written. The gets and puts function require the address of a character array as an argument. When a character array for a string is declare it must be having space to accomodate for the null character.
- The syntax of puts function is
`puts(variable-name)`

⇒) Program:-

```
#include <stdio.h>
#include <string.h>
int main ()
{
    char name[20];
    printf("Enter your name");
    gets(name);
    puts(name);
    return 0;
}
```

O/P:- Enter your name.

Files in C:-

→ C support the concept of file through which the data can be store in the Flopidisc or any others secondary storage devices. So the data can be access when required a file is a collection of data or text, placed on the disc or any other secondary storage devices. There are two types of files.

- ① Sequential access file
- ② Random access file

① Sequential access:-

→ In sequential file data are kept sequentially for example if we want to access the 58th record then would be read sequentially to reach the 58th record it means it will be access sequentially.

② Random access:-

→ In the random access file the data can be access and process randomly. In this case the 58 records can be access directly. In this case accessing the record takes less time than the sequential file.

* Basic File operation in C:-

- (i) Naming a file
- (ii) opening a file
- (iii) Read data from a file
- (iv) Writing data in file
- (v) close file

(i) Naming a file:-

→ Naming a files means to give a particular name to a file using character string. When we store a record in the file then first of all we require temporary area in a memory. For storing this record in the memory we use the pointer which points the starting address where this record are store.

Syntax:- `FILE *P;`

→ Where `p` is a pointer of type file.

(ii) Opening a file:-

→ For opening a file we use the library function `fopen()`. First we declare pointer variable and `fopen()` as file type pointer.

Syntax:- `FILE *P *fopen();`

(iii) Read data from a file:-

→ For read and write of a file we use the following syntax is

$$p = \text{fopen}(\text{"File-name"}, \text{"mode"});$$

→ These are the six types of modes:-

(i) W (ii) a (iii) r
(iv) W+ (v) a+ (vi) r+

(i) W (write) mode:-

→ This mode opens a new file for writing a record. If the file name already exists then this mode erase the previous record and the new record enter are written to the file.

syntax:- $p = \text{fopen}(\text{"file-name"}, \text{w});$

(ii) a (append):-

→ This mode open a file for appending the data or record. If the file does not exist then the task of this mode is same as (w) mode.

syntax:- $p = \text{fopen}(\text{"file-name"}, \text{a});$

(iii) r (read):-

→ This mode opens a file for reading purpose only. If the file record does not exist then compiler returns null to the file pointer.

syntax:- $p = \text{fopen}(\text{"file-name"}, \text{r});$

(iv) w+ (write + Read) :-

→ This mode is used both for reading and writing purpose this is same as the w mode but we can also read the record which is store in the file.

Syntax: `p = fopen("file - name", wt);`

(v) a+ (append + Read)

→ This mode is used both for read and appending the record. This mode is same as the mode a but we can also read the record which is store in the file.

Syntax: `p = fopen("file - name", at);`

(vi) r+ (read + write)

→ This mode is used both for reading and writing purpose. we can read the record and also write the record in the file. from this mode the previous record of the file are not deleted.

Syntax: `p = fopen("file - name", r+);`

(iv) close file:-

→ The files which are opened from the fopen function must be close at the end of the program

fclose;

fclose:-

`fclose(p);`

⇒ Syntax of file program:-

```
int main()
{
    FILE *P, *fopen();
    P = fopen("filename", "mode");
```

```
fclose(P);
}
```

Text File :-

→ A text file contains data in the form of ASCII character and it generally use to store a stream of characters.

- 1) Each line in a text file ends with a new line character. \n.
- 2) It can be read or written by any text editor.
- 3) They are generally store with (.txt) file extension.
- 4) Text file also can be used to store the source code.

Binary File :-

→ A binary file contains data in binary form instead of ASCII character they contain data that is store in a similar manner to how it is store in main memory.

- 1) The binary file can be created only from within a program and their constants can

only be read by a program.

- 2) More secure as they are easily readable.
- 3) They are generally store with (.bin) file extension.

INPUT / OUTPUT Functions:- For File

(i) → fprintf() :-

Syntax: `fprintf(fp, "control character", variable-names)`

Program: `#include <iostream.h>`

`int main()`

`{`

`FILE *p, *fopen();`

`char name[30];`

`p = fopen("zero2.dat", "w");`

`printf("Enter the name");`

`scanf("%s", &name);`

`fprintf(p, "My name is %s", name);`

`fclose(p);`

`}`

Enter name

ABC

My name is ABC

→ This function is same as the printf function but it writes the data into the file. so it has one more parameter that is the file pointer.

(ii) fscanf:-

→ This function is same as the scanf function but this reads the data from the file so this has more parameter like the file pointer.

fscanf(fptr, "control character", &variable name);

(iii) getc:-

→ This is macro which is used to read a single character from a given file and increment the file pointer. If the file pointer reaches the end of the file then it returns an error is counted.

Syntax:- getc(fptr);

Ex:- ch = getc(fptr);

fptr = file pointer

ch = variable

(iv) putc:-

→ This is macro which is used to write a single character into a file. If an error occurs it return the value eof (error on file).

Syntax: putc(ch, fptr);

(v) fgetc:-

→ The fgetc is also used to read a single character from a given file and error is encountered.

→ fgetc is a true function available in the stdio.h library.

Syntax:- fgetc(fptr);

Ex:- ch = fgetc(fptr);

(vi) fputc:-

→ The fputc function is also used to write a single character into a given file. The syntax of fputc is same as the putc function. It is a true function available in the library function stdio.h.

Syntax:- fputc(ch, fptr);

(vii) fgets():-

→ fgets is used to read a set of characters as a string from a given file and copy is that string to a memory location which is reference by an array.

Syntax:- fgets(sptr, max, fptr);

(viii) fputs():-

→ The fput function is used to write a string to a given file. It is generally used to copy string from one file to another file.

Syntax:- fputs(sptr, fptr);

(ix) fread:-

→ This function is used to read an entire block of binary data from a given file. It is also used to read one or more structure from a given file and copy's them to a given memory location.

Syntax:- `fread(ptr, size, n, fptr);`

(x) fwrite():

→ This function is used to write an entire block of binary data to a given file. It is also used to write one or more structure to a given file.

Syntax:- `fwrite(ptr, size, n, fptr);`

(xi) feof(): end of file.

→ This function is used to detect end of the file. It returns non zero value. If the file pointer is at the end of the file otherwise it returns 0.

Syntax:- `feof(fptr);`

(xii) ferror():

→ This function is used to test the given file for an error. It returns non-zero value only, if error occurs. otherwise it returns 0.

Syntax:- `ferror(fptr);`

(xiii) `fflush()` :-

→ This function is used to empty the buffer of a given file immediately after writing the buffer contents to the file.

Syntax :- `fflush (fptr);`

Positioning Function:- (Random Accessing Fun^c):-

File status

→ Normally the files are read sequentially. C language support the method which give us freedom from accessing the record sequentially. The following fun^c are used in the file processor.

(i) fseek()

→ This function is used for setting the pointer position in the file at the desired location.

→ The syntax is

$fseek(fptr, offset, ptr - position);$

→ Where $fptr$ is a pointer to the file pointer, $offset$ is a number positive for forward and negative for backward which indicates the no. of position in bytes to be move from the location specified by the pointer position and $ptr - position$ is an integer which can tag anyone of the following values.

Value	Meaning
0	Beginning of file
1	Current position
2	End of file

Ex:- $fseek(fptr, 0L, 0) =$ Go to beginning of file
 $fseek(fptr, 10L, 0) =$ move 10 bytes forward from beginning of file.

$fseek(fptr, 4L, 1) =$ Move 4 bytes from current position
 $fseek(fptr, -4L, 2) =$ move 4 bytes backward from end of the file.

(ii) `f tell ()` :-

- This function written the value of the current pointer position in the file. The syntax is
`p = f tell (FPtr);`
- Where `FPtr` is a pointer to the file and `p` is the variable to receive the position which is always a long integer.

⇒ Program:-

```
#include <stdio.h>
int main()
{
    FILE *FPtr;
    long int n;
    FPtr = fopen("notes.txt", "r");
    fseek(FPtr, 0L, 2);
    n = f tell (FPtr);
    printf("No. of character %d", n);
    fclose(FPtr);
    return 0;
}
```

(iii) `rewind ()` :-

- This function is used to move the file pointer to the beginning of the given file.
- The syntax is `rewind(FPtr);`
- Where `FPtr` is a pointer to the file.